

Chapter 2-3

Geometric Image Transform
Spatial Transform

2.x.1 Fundamentals

- 2.x.1.1 Spatial Transform
 - Input image: $f(u,v)$
 - Output image: $g(x,y)$
 - $g(x,y) = f(u,v)$
 - $(x,y) = T[(u,v)],$

2.x.1 Fundamentals

- 2.x.1.2 Forward Mapping

$$(x,y) = (X(u,v),Y(u,v))$$

example: Zoom out

$$(x,y) = (u/2,v/2)$$

[Programming]

2.x.1 Fundamentals

- 2.x.1.3 Inverse Mapping

$$(u,v) = (U(x,y),V(x,y))$$

example: Zoom out

$$(u,v) = (x*2,y*2)$$

[Programming]

2.x.1 Fundamentals

- example: Zoom in
FM: $(x,y) = (u*2,v*2)$
IM : $(u,v) = (x/2,y/2)$

[Programming]

2.x.1 Fundamentals

- Usually
 - Image Processing(Spatial Transform) using IM
 - Computer Graphics (Geometrics Render) using FM

2.x.2 Affine Transform

- 2.x.2.1 Definition of Affine Transform

$$A(x) = L(x) + c$$

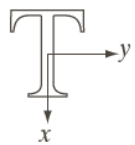
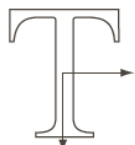
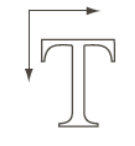
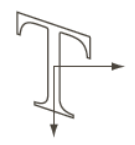
- 2.x.2.2 General Form

$$\begin{bmatrix} x & y & 1 \end{bmatrix} = \begin{bmatrix} u & v & 1 \end{bmatrix} T = \begin{bmatrix} u & v & 1 \end{bmatrix} \begin{bmatrix} t_{11} & t_{12} & 0 \\ t_{21} & t_{22} & 0 \\ t_{31} & t_{32} & 1 \end{bmatrix}$$

2.x.2 Affine Transform

TABLE 2.2

Affine transformations based on Eq. (2.6.–23).

Transformation Name	Affine Matrix, T	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = v$ $y = w$	
Scaling	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = c_x v$ $y = c_y w$	
Rotation	$\begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = v \cos \theta - w \sin \theta$ $y = v \sin \theta + w \cos \theta$	
Translation	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$	$x = v + t_x$ $y = w + t_y$	
Shear (vertical)	$\begin{bmatrix} 1 & 0 & 0 \\ s_v & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = v + s_v w$ $y = w$	
Shear (horizontal)	$\begin{bmatrix} 1 & s_h & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x = v$ $y = s_h v + w$	

2.x.2 Affine Transform

- 2.x.2.3 Others **Exercise**

- Flip Transform

- Flip Horizontal
- Flip Vertical

- Zoom At (x_0, y_0)

- Local Zooming

[Programming]

Exercise

- FlipV



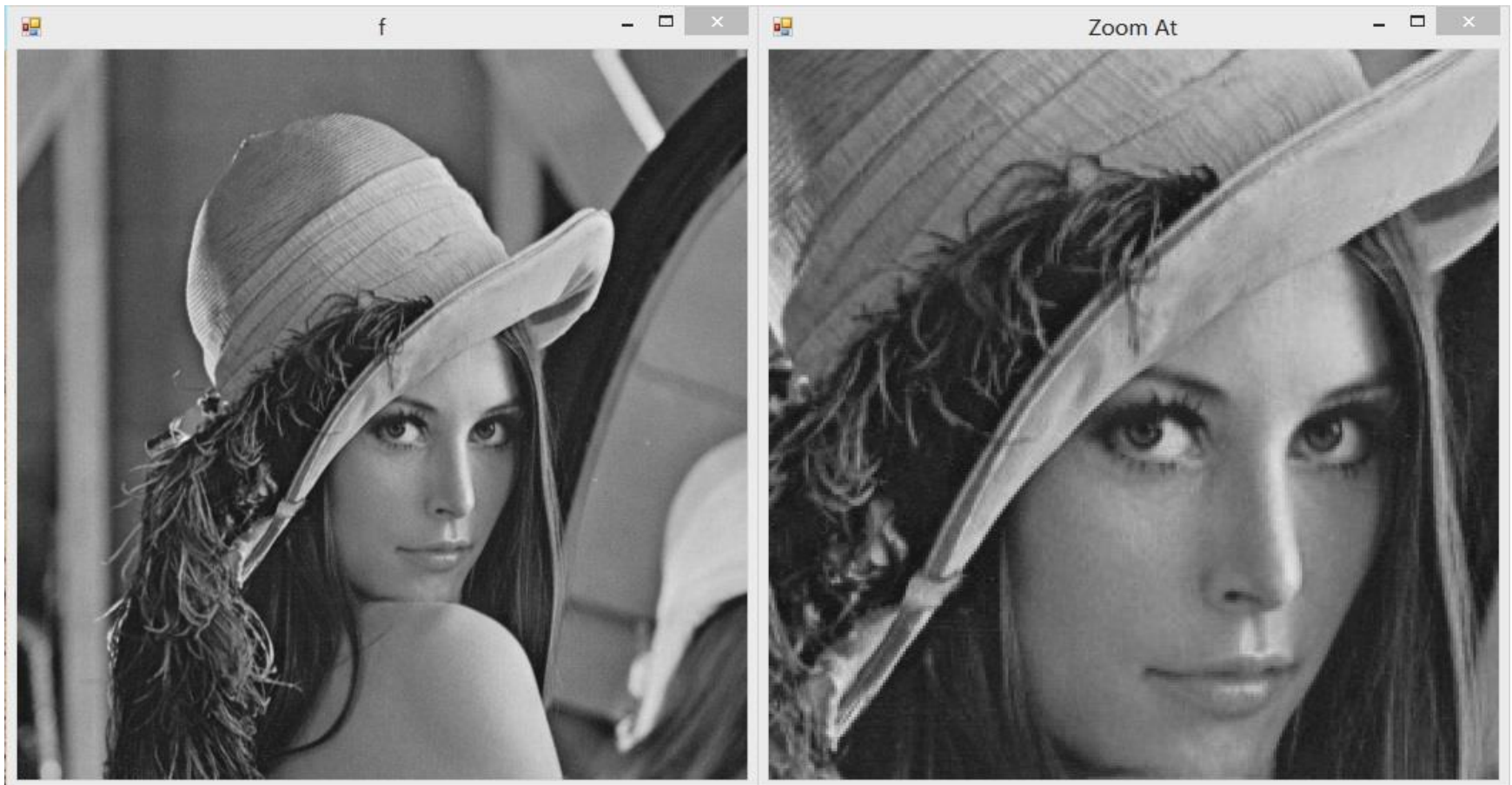
Exercise

- FlipH



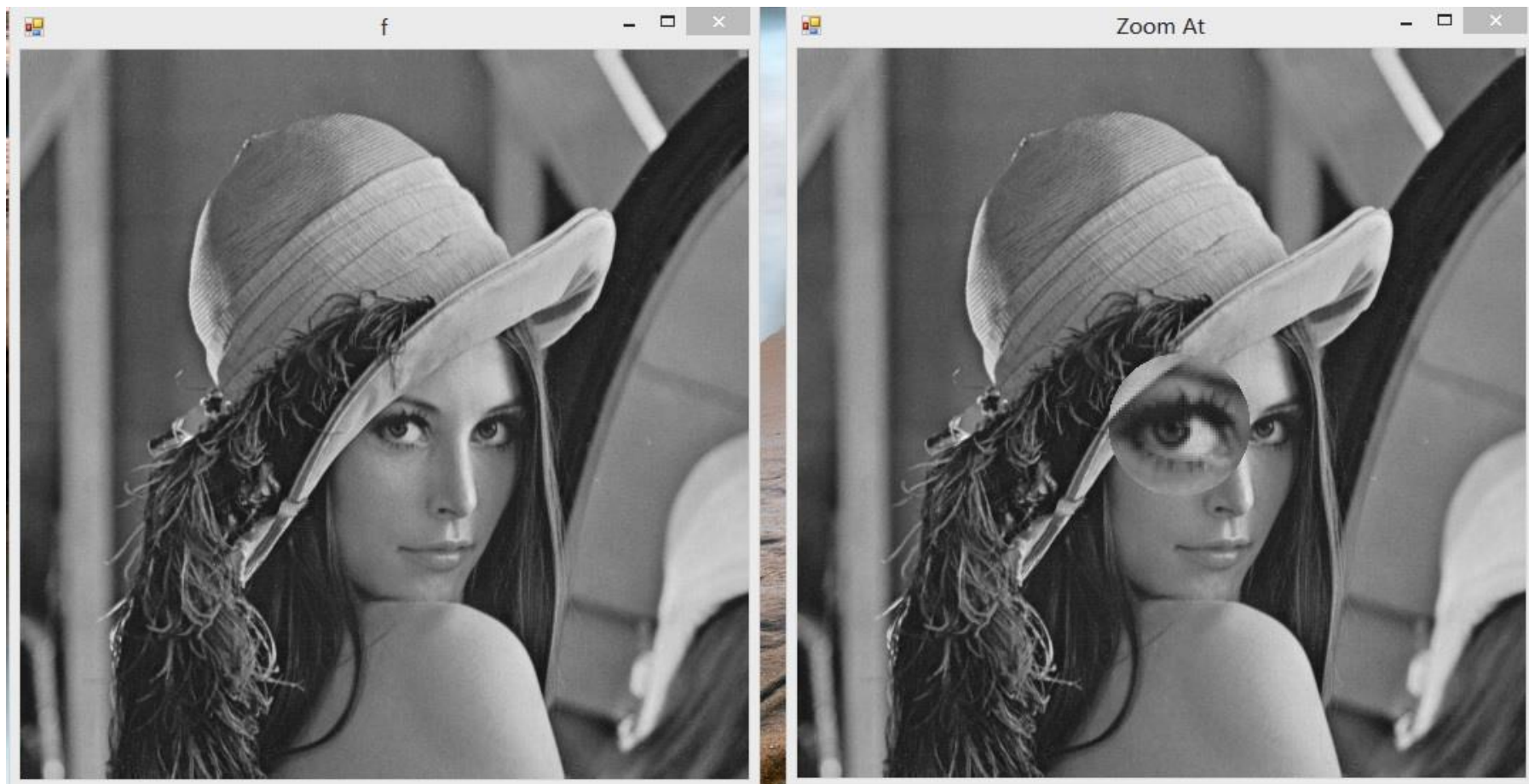
Exercise

- `byte[,] ZoomAt(byte[,]f,int x0,int y0,float K)`



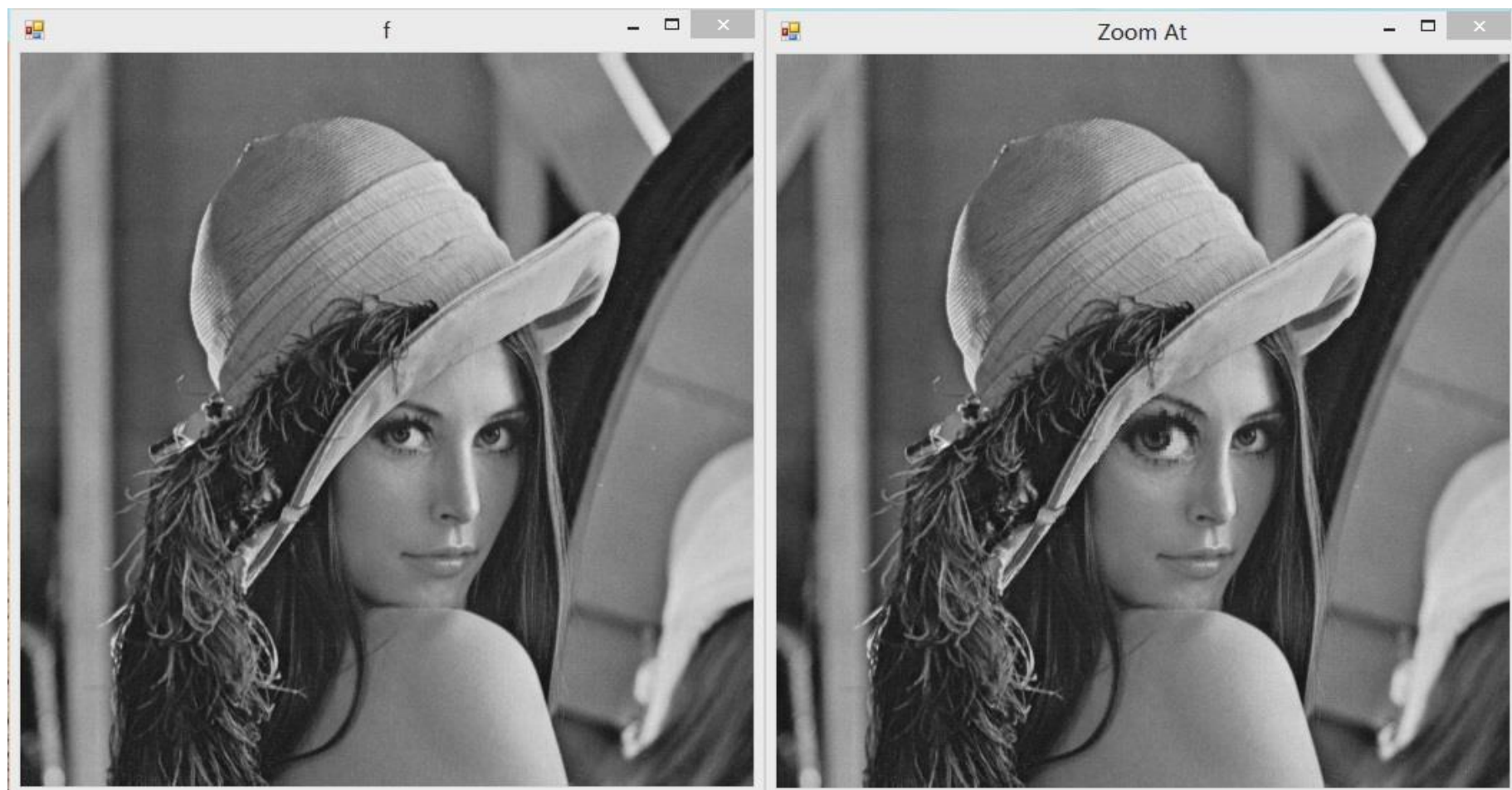
Exercise

- `byte[,] localZooming(byte[,]f,int x0,int y0,float R,float K)`



Exercise

- `byte[,] localZooming2(byte[,]f,int x0,int y0,float R,float K)`



Composted Affine Transform

- Basic Affine Transform
 - Translation, Scaling, Rotation
- Composted Affine Transform
 - Zooming At

Affine Transform Properties

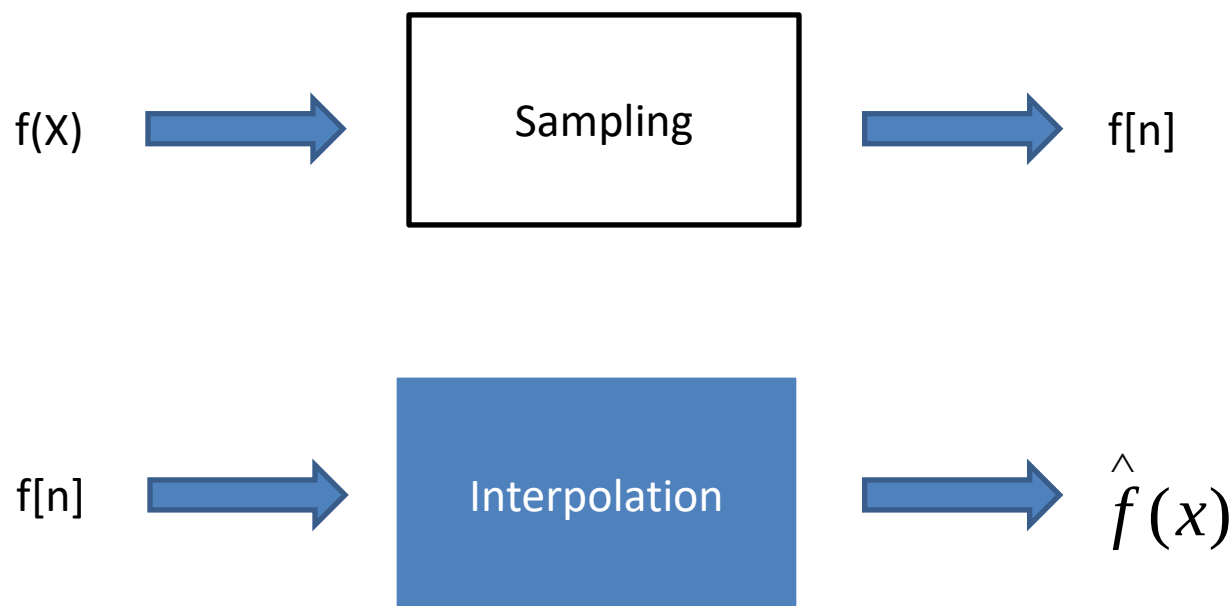
- Composed Affine Transform is Affine Transform
- Inverse Affine Transform is Affine Transform

2.x.3 Image Interpolation

- 2.x.3.1 Background
 - [Programming of Zoom in 4X4]

2.x.3 Image Interpolation

- 2.x.3.2 Sampling & Interpolation



2.x.3 Image Interpolation

- 2.x.3.2 Sampling & Interpolation

$$f[n] = f(n) \quad , \quad f[n] = \text{int}(f(n))$$

$$\hat{f}(x) = \sum_n f(n) \cdot h(x - n)$$

$h(x)$: Interpolation kernel

2.x.3 Image Interpolation

- 2.x.3.2 Nearest Neighborhood Interpolation

$$h(x) = \begin{cases} 1, & |x| < 0.5 \\ 0, & \text{others} \end{cases}$$

– 2D: $x' = (\text{int})(x+0.5), y' = (\text{int})(y+0.5)$

2.x.3 Image Interpolation

- 2.x.3.2 Bilinear Interpolation

$$h(x) = \begin{cases} 1+x, & -1 \leq x < 0 \\ 1-x, & 0 \leq x \leq 1 \\ 0, & |x| > 1 \end{cases}$$

- $i \leq x < i+1$
- $a = x - i$
 - $f(x) = f[i] * (1-a) + f[i+1] * a;$

2.x.3 Image Interpolation

- 2.x.3.2 Bilinear Interpolation

- 2D

$$i \leq x < i+1, j \leq y < j+1,$$

$$a = x - i, b = y - j$$

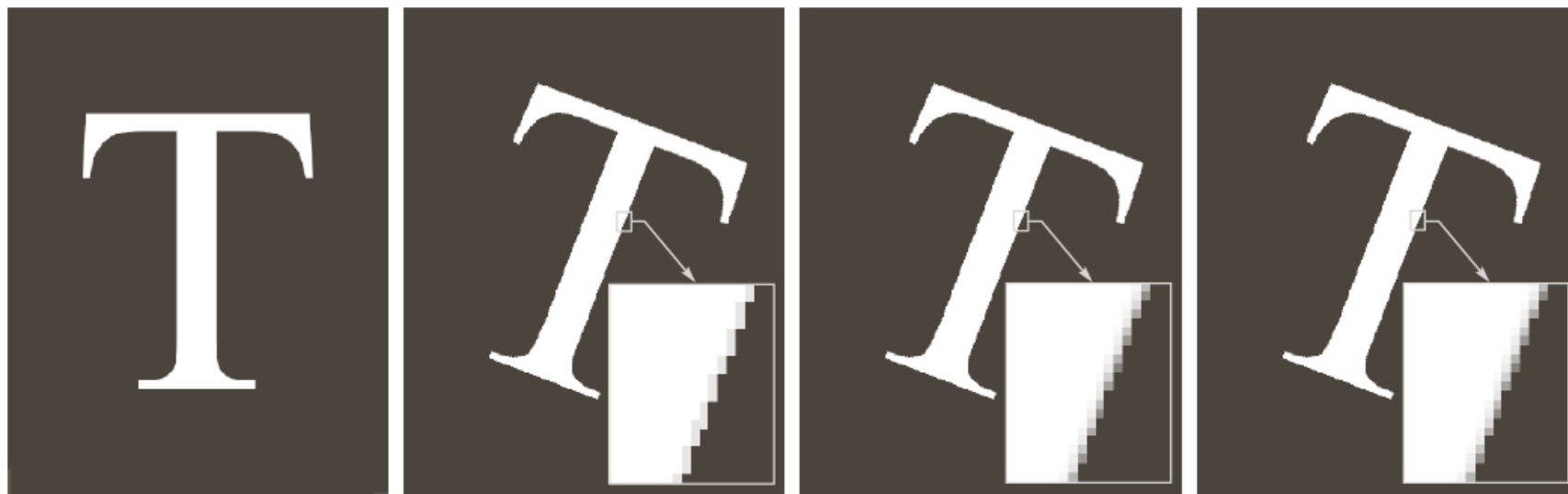
$$- f(x, j) = f[i, j] * (1 - a) + f[i + 1, j] * a;$$

$$- f(x, j + 1) = f[i, j + 1] * (1 - a) + f[i + 1, j + 1] * a;$$

$$- f(x, y) = f(x, j) * (1 - b) + f(x, j + 1) * b;$$

[Programming]

2.x.3 Image Interpolation



a b c d

FIGURE 2.36 (a) A 300 dpi image of the letter T. (b) Image rotated 21° clockwise using nearest neighbor interpolation to assign intensity values to the spatially transformed pixels. (c) Image rotated 21° using bilinear interpolation. (d) Image rotated 21° using bicubic interpolation. The enlarged sections show edge detail for the three interpolation approaches.

Image Registration

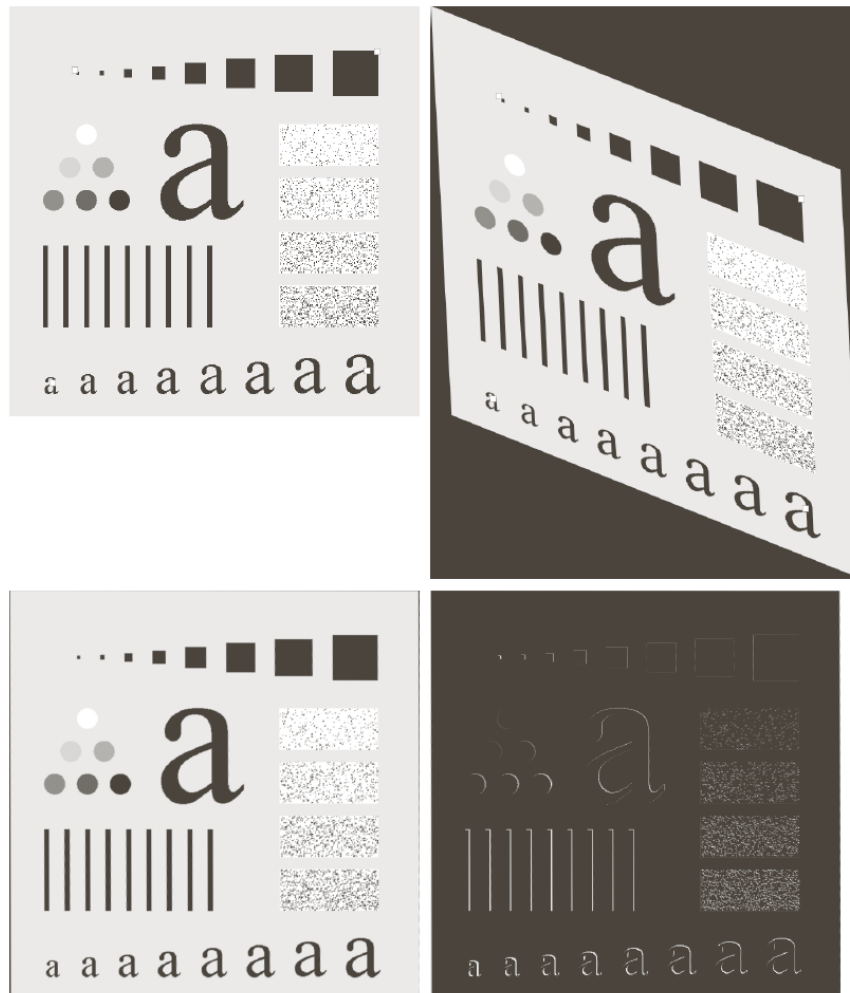


FIGURE 2.37

Image registration. (a) Reference image. (b) Input (geometrically distorted image). Corresponding tie points are shown as small white squares near the corners. (c) Registered image (note the errors in the borders). (d) Difference between (a) and (c), showing more registration errors.